

第一部分：理论篇

1.1 为什么要学文本处理？

Linux 是一个"一切皆文件"的系统。配置文件、日志文件、数据文件都是文本。运维工作中大量时间都在和文本打交道：查日志、改配置、提取数据、批量替换。

文本处理三剑客就是处理这些任务的利器。

1.2 三剑客概览

工具		核心功能	擅长场景
grep		搜索过滤	从大量文本中找到匹配的行
sed		流编辑	替换、删除、插入文本
awk		文本分析	按列处理、统计计算、格式化输出

通俗比喻：

工具	比喻	说明
grep	搜索引擎	在书中找包含某个词的所有页
sed	批量替换	Word 中的"查找替换"功能
awk	Excel	按列处理数据，做统计分析

1.3 什么是正则表达式？

正则表达式（Regular Expression，简称 regex）是一种描述文本模式的语言。它不是匹配固定的字符串，而是匹配符合某种规则的字符串。

生活化例子：

需求	固定字符串	正则表达式
找"error"	只能找 error	只能找 error
找所有错误	要写 error, Error, ERROR...	<code>[Ee]rror</code> 或 <code>(?i)error</code>
找任意 IP 地址	要列出所有可能的 IP? 不现实	<code>\d+\.\d+\.\d+\.\d+</code>
找所有手机号	不可能	<code>1[3-9]\d{9}</code>

正则表达式让你能用一个模式匹配无数种可能。

1.4 基本正则表达式（BRE）元字符

Linux 中默认使用基本正则表达式（Basic Regular Expression，BRE）。

1.4.1 字符匹配

元字符	含义	示例	匹配
<code>.</code>	匹配任意单个字符	<code>a.c</code>	abc, adc, a1c, a c
<code>[]</code>	匹配括号内任意一个字符	<code>[abc]</code>	a 或 b 或 c
<code>[^]</code>	匹配不在括号内的字符	<code>[^abc]</code>	除 a, b, c 外的任意字符
<code>[a-z]</code>	匹配范围内的字符	<code>[a-z]</code>	任意小写字母
<code>[0-9]</code>	匹配数字	<code>[0-9]</code>	任意数字

1.4.2 位置锚定

元字符	含义	示例	匹配
<code>^</code>	行首	<code>^hello</code>	以 hello 开头的行
<code>\$</code>	行尾	<code>world\$</code>	以 world 结尾的行
<code>^\$</code>	空行	<code>^\$</code>	完全空白的行
<code>\<</code>	词首	<code>\<the</code>	the, them (不匹配 other)
<code>\></code>	词尾	<code>the\></code>	the, bathe (不匹配 them)
<code>\b</code>	词边界	<code>\bthe\b</code>	只匹配独立的 the

1.4.3 次数匹配

元字符	含义	示例	匹配
<code>*</code>	前一个字符出现 0 次或多次	<code>ab*c</code>	ac, abc, abbc, abbbc...
<code>\+</code>	前一个字符出现 1 次或多次	<code>ab\+c</code>	abc, abbc... (不匹配 ac)
<code>\?</code>	前一个字符出现 0 次或 1 次	<code>ab\?c</code>	ac, abc
<code>\{n\}</code>	前一个字符恰好出现 n 次	<code>a\{3\}</code>	aaa
<code>\{n,\}</code>	前一个字符至少出现 n 次	<code>a\{2,\}</code>	aa, aaa, aaaa...
<code>\{n,m\}</code>	前一个字符出现 n 到 m 次	<code>a\{2,4\}</code>	aa, aaa, aaaa

1.4.4 分组与引用

元字符	含义	示例	说明
<code>\(\)</code>	分组	<code>\(ab\)*</code>	ab 整体重复 0 次或多次
<code>\1</code>	反向引用第 1 个分组	<code>\(.\)\1</code>	匹配连续两个相同字符
<code>\2</code>	反向引用第 2 个分组	见 sed 实例	

1.5 扩展正则表达式（ERE）

扩展正则表达式（Extended Regular Expression, ERE）语法更简洁，不需要那么多反斜杠。

使用方式：`grep -E` 或 `egrep`

BRE（基本）	ERE（扩展）	含义
<code>\+</code>	<code>+</code>	1 次或多次
<code>\?</code>	<code>?</code>	0 次或 1 次
<code>\{n,m\}</code>	<code>{n,m}</code>	n 到 m 次
<code>\(\)</code>	<code>()</code>	分组
<code> </code>	<code> </code>	

建议：使用扩展正则更清晰，养成用 `grep -E` 和 `sed -r` 的习惯。

1.6 常用字符类

POSIX 定义了一些字符类，可在 `[ ]` 内使用：

字符类	等价于	含义
<code>[[[:alpha:]]]</code>	<code>[a-zA-Z]</code>	字母
<code>[[[:digit:]]]</code>	<code>[0-9]</code>	数字
<code>[[[:alnum:]]]</code>	<code>[a-zA-Z0-9]</code>	字母和数字
<code>[[[:space:]]]</code>	空白字符	空格、制表符、换行等
<code>[[[:upper:]]]</code>	<code>[A-Z]</code>	大写字母
<code>[[[:lower:]]]</code>	<code>[a-z]</code>	小写字母
<code>[[[:punct:]]]</code>	标点符号	标点符号

1.7 贪婪匹配与非贪婪匹配

正则表达式默认是贪婪的，会尽可能多地匹配。

```
文本: <title>Hello</title>
正则: <.*>
匹配: <title>Hello</title>  ← 整个都匹配了
```

基本正则不支持非贪婪模式。如需非贪婪，可以用其他方式：

```
正则: <[^>]*>
匹配: <title>  ← 只匹配到第一个 >
```

第二部分：实践篇 – grep

2.1 grep 基础

grep 用于搜索文本，打印匹配的行。

基本语法：

```
grep [选项] '模式' 文件
```

准备测试文件：

```
# 创建测试文件
cat > /tmp/test.txt << 'EOF'
Hello World
hello world
HELLO WORLD
This is a test line.
This is line 2.
There are 100 apples.
Error: file not found
error: permission denied
ERROR: system failure
192.168.1.1 - - [01/Jan/2024:10:00:00] "GET /index.html"
192.168.1.2 - - [01/Jan/2024:10:00:01] "POST /api/login"
10.0.0.1 - - [01/Jan/2024:10:00:02] "GET /admin"

root:x:0:0:root:/root:/bin/bash
nginx:x:101:101:nginx:/var/www:/sbin/nologin
mysql:x:102:102:mysql:/var/lib/mysql:/sbin/nologin
EOF
```

2.2 grep 常用选项

选项	作用	示例
<code>-i</code>	忽略大小写	<code>grep -i 'error' file</code>
<code>-v</code>	反向匹配（不包含）	<code>grep -v 'error' file</code>
<code>-n</code>	显示行号	<code>grep -n 'error' file</code>
<code>-c</code>	只统计匹配行数	<code>grep -c 'error' file</code>
<code>-l</code>	只显示匹配的文件名	<code>grep -l 'error' *.log</code>
<code>-L</code>	只显示不匹配的文件名	<code>grep -L 'error' *.log</code>
<code>-w</code>	匹配整个单词	<code>grep -w 'the' file</code>
<code>-x</code>	匹配整行	<code>grep -x 'hello' file</code>
<code>-o</code>	只输出匹配部分	<code>grep -o '[0-9]\+' file</code>
<code>-E</code>	使用扩展正则	<code>grep -E 'a+b' file</code>
<code>-A n</code>	显示匹配行及后 n 行	<code>grep -A 3 'error' file</code>
<code>-B n</code>	显示匹配行及前 n 行	<code>grep -B 3 'error' file</code>
<code>-C n</code>	显示匹配行及前后各 n 行	<code>grep -C 3 'error' file</code>
<code>-r</code>	递归搜索目录	<code>grep -r 'error' /var/log/</code>
<code>-q</code>	静默模式，不输出	<code>grep -q 'error' file && echo found</code>

2.3 grep 实战练习

练习 1：基础搜索

```
# 搜索包含 error 的行（区分大小写）
grep 'error' /tmp/test.txt

# 搜索包含 error 的行（不区分大小写）
grep -i 'error' /tmp/test.txt

# 显示行号
grep -in 'error' /tmp/test.txt

# 统计匹配行数
grep -ic 'error' /tmp/test.txt
```

练习 2：反向匹配

```
# 显示不包含 error 的行
grep -v 'error' /tmp/test.txt

# 显示不包含 error（忽略大小写）的行
grep -iv 'error' /tmp/test.txt
```

练习 3：正则表达式匹配

```
# 匹配以 hello 开头的行（忽略大小写）
grep -i '^hello' /tmp/test.txt

# 匹配以数字结尾的行
grep '[0-9]$' /tmp/test.txt

# 匹配空行
grep '^$' /tmp/test.txt

# 匹配非空行
grep -v '^$' /tmp/test.txt

# 匹配包含数字的行
grep '[0-9]' /tmp/test.txt
```



```
# 匹配 IP 地址 (简单版)
grep -E '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' /tmp/test.txt

# 只输出匹配的 IP 地址
grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' /tmp/test.txt
```

练习 4：单词边界

```
# 创建测试文件
echo -e "the\nthere\nother\nbathe" > /tmp/words.txt

# 匹配包含 the 的行 (会匹配所有)
grep 'the' /tmp/words.txt

# 只匹配独立的单词 the
grep -w 'the' /tmp/words.txt

# 使用词边界
grep '\bthe\b' /tmp/words.txt
grep '<the>' /tmp/words.txt
```

练习 5：上下文显示

```
# 创建日志文件
cat > /tmp/app.log << 'EOF'
2024-01-01 10:00:00 INFO Starting application
2024-01-01 10:00:01 INFO Loading configuration
2024-01-01 10:00:02 DEBUG Config loaded successfully
2024-01-01 10:00:03 ERROR Database connection failed
2024-01-01 10:00:04 ERROR Retrying connection...
2024-01-01 10:00:05 INFO Connected to database
2024-01-01 10:00:06 INFO Application started
EOF

# 显示 ERROR 及其后 2 行
grep -A 2 'ERROR' /tmp/app.log

# 显示 ERROR 及其前 2 行
```

```
grep -B 2 'ERROR' /tmp/app.log
```

显示 ERROR 及其前后各 1 行

```
grep -C 1 'ERROR' /tmp/app.log
```

练习 6：递归搜索

在 /etc 目录下搜索包含 root 的文件

```
grep -r 'root' /etc/ 2>/dev/null | head -10
```

只显示文件名

```
grep -rl 'root' /etc/ 2>/dev/null | head -10
```

指定文件类型

```
grep -r --include='*.conf' 'root' /etc/ 2>/dev/null | head -10
```

练习 7：结合管道

从 ps 输出中查找 nginx 进程

```
ps aux | grep nginx
```

排除 grep 自身

```
ps aux | grep nginx | grep -v grep
```

更好的方式

```
ps aux | grep '[n]ginx'
```

从日志中统计 ERROR 数量

```
cat /tmp/app.log | grep -c 'ERROR'
```

提取日志中的时间

```
grep -oE '[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}'  
/tmp/app.log
```

第三部分：实践篇 – sed

3.1 sed 基础

sed 是流编辑器，逐行读取文本，按指定规则处理，然后输出。

基本语法：

```
sed [选项] '动作' 文件
```

核心特点：

sed 默认不修改原文件，只是把处理结果输出到屏幕。要修改原文件需要用 `-i` 选项。

3.2 sed 工作流程

```
读取一行 → 存入模式空间 → 执行命令 → 输出模式空间 → 清空模式空间 → 读取下一行
```

3.3 sed 常用选项

选项	作用
<code>-n</code>	静默模式，不自动输出，需配合 p 命令
<code>-e</code>	指定多个编辑命令
<code>-i</code>	直接修改原文件（危险！）
<code>-i.bak</code>	修改原文件并备份
<code>-r</code>	使用扩展正则表达式
<code>-f</code>	从文件读取 sed 脚本

3.4 sed 地址定位

sed 可以指定对哪些行进行操作：

地址形式	含义	示例
n	第 n 行	3 表示第 3 行
\$	最后一行	\$
n,m	第 n 到 m 行	2,5 表示第 2-5 行
n,+m	第 n 行及其后 m 行	2,+3 表示第 2-5 行
n~m	从第 n 行开始，每隔 m 行	1~2 表示奇数行
/pattern/	匹配模式的行	/error/
/pattern1/,/pattern2/	从匹配 pattern1 到 pattern2 的行	/start/,/end/

3.5 sed 常用命令

命令	作用	示例
p	打印	<code>sed -n '3p' file</code>
d	删除	<code>sed '3d' file</code>
s	替换	<code>sed 's/old/new/' file</code>
a	在行后追加	<code>sed '3a new line' file</code>
i	在行前插入	<code>sed '3i new line' file</code>
c	替换整行	<code>sed '3c new line' file</code>
y	字符转换	<code>sed 'y/abc/ABC/' file</code>
=	打印行号	<code>sed '=' file</code>
r	读取文件内容插入	<code>sed '3r other.txt' file</code>
w	将匹配行写入文件	<code>sed -n '/error/w err.txt' file</code>

3.6 sed 替换命令详解

替换是 sed 最常用的功能：

```
sed 's/模式/替换内容/标志' 文件
```

替换标志：

标志	作用
<code>g</code>	替换行内所有匹配（默认只替换第一个）
<code>i</code>	忽略大小写
<code>p</code>	打印替换后的行
<code>w file</code>	将替换后的行写入文件
数字	只替换第 n 个匹配

3.7 sed 实战练习

练习 1：打印指定行

```
# 创建测试文件
cat > /tmp/sed_test.txt << 'EOF'
line 1: apple
line 2: banana
line 3: cherry
line 4: date
line 5: elderberry
line 6: fig
line 7: grape
EOF

# 打印第 3 行
sed -n '3p' /tmp/sed_test.txt

# 打印第 2 到 5 行
sed -n '2,5p' /tmp/sed_test.txt

# 打印最后一行
sed -n '$p' /tmp/sed_test.txt

# 打印奇数行
sed -n '1~2p' /tmp/sed_test.txt
```

```
# 打印偶数行
sed -n '2~2p' /tmp/sed_test.txt

# 打印包含 a 的行
sed -n '/a/p' /tmp/sed_test.txt
```

练习 2：删除行

```
# 删除第 3 行
sed '3d' /tmp/sed_test.txt

# 删除第 2 到 5 行
sed '2,5d' /tmp/sed_test.txt

# 删除包含 cherry 的行
sed '/cherry/d' /tmp/sed_test.txt

# 删除空行
sed '/^$/d' /tmp/test.txt

# 删除以 # 开头的注释行
sed '/^#/d' /tmp/test.txt

# 删除空行和注释行
sed '/^$/d; /^#/d' /tmp/test.txt
# 或
sed -e '/^$/d' -e '/^#/d' /tmp/test.txt
```

练习 3：基本替换

```
# 替换第一个匹配
echo "hello hello hello" | sed 's/hello/hi/'

# 替换所有匹配
echo "hello hello hello" | sed 's/hello/hi/g'

# 只替换第 2 个匹配
echo "hello hello hello" | sed 's/hello/hi/2'
```

```
# 替换并忽略大小写
echo "Hello HELLO hello" | sed 's/hello/hi/gi'

# 使用不同的分隔符（当内容包含 / 时）
echo "/usr/local/bin" | sed 's#/usr/local#/opt#'
echo "/usr/local/bin" | sed 's@/usr/local@/opt@'
```

练习 4：使用正则替换

```
# 删除行首空白
echo "    hello world" | sed 's/^[[:space:]]*//'

# 删除行尾空白
echo "hello world    " | sed 's/[[:space:]]*$//'

# 删除行首行尾空白
echo "    hello world    " | sed 's/^[[:space:]]*//; s/[[:space:]]*$//'

# 将多个空格替换为一个
echo "hello    world" | sed 's/  */ /g'

# 删除所有空格
echo "hello world" | sed 's/ //g'

# 在每行开头添加字符
sed 's/^/# /' /tmp/sed_test.txt

# 在每行结尾添加字符
sed 's$/ ---/' /tmp/sed_test.txt
```

练习 5：分组与引用

```
# 交换两个单词
echo "hello world" | sed -r 's/([a-z]+) ([a-z]+)/\2 \1/'

# 给数字加上括号
echo "There are 100 apples" | sed -r 's/([0-9]+)/[\1]/g'
```



```
# 提取并重新格式化日期
echo "2024-01-15" | sed -r 's/([0-9]{4})-([0-9]{2})-([0-9]{2})/\3\/\2\/\1/'
# 输出: 15/01/2024

# 将用户名和 UID 交换
echo "root:x:0:0" | sed -r 's/^([^:]+):([^:]+):([^:]+):/\3:\2:\1:/'

# 匹配重复字符
echo "bookkeeper" | sed -r 's/(.)\1/[\1\1]/g'
```

练习 6：插入和追加

```
# 在第 3 行后追加
sed '3a This is a new line' /tmp/sed_test.txt

# 在第 3 行前插入
sed '3i This is a new line' /tmp/sed_test.txt

# 在匹配行后追加
sed '/cherry/a --- after cherry ---' /tmp/sed_test.txt

# 替换整行
sed '3c This line is replaced' /tmp/sed_test.txt

# 追加多行（使用 \n 或反斜杠）
sed '3a line A\nline B\nline C' /tmp/sed_test.txt
```

练习 7：实际应用场景

```
# 场景 1：修改配置文件
# 创建模拟配置文件
cat > /tmp/config.conf << 'EOF'
# Database Configuration
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
```

```
DB_PASS=password123
# Server Configuration
SERVER_PORT=8080
EOF

# 修改数据库主机
sed -i.bak 's/DB_HOST=.* /DB_HOST=192.168.1.100/' /tmp/config.conf

# 修改端口（只匹配 DB_PORT）
sed -i 's/^DB_PORT=.* /DB_PORT=3307/' /tmp/config.conf

# 查看结果
cat /tmp/config.conf

# 场景 2：批量注释配置
# 注释掉某些行
sed -i 's/^DB_PASS/#&/' /tmp/config.conf

# 取消注释
sed -i 's/^#(DB_PASS\)/\1/' /tmp/config.conf

# 场景 3：处理日志
# 提取 ERROR 行并去掉时间戳
grep 'ERROR' /tmp/app.log | sed -r 's/^[0-9-]+ [0-9:]+//'

# 场景 4：处理 /etc/passwd
# 显示用户名和 shell
sed -r 's/^([:]+).*:([:]+)$/\1 -> \2/' /etc/passwd | head -5
```

练习 8：sed 脚本文件

```
# 创建 sed 脚本
cat > /tmp/process.sed << 'EOF'
# 删除空行
/^$/d
# 删除注释行
/^#/d
# 去掉行首空格
s/^[[:space:]]*//
# 将 localhost 替换为实际 IP
```

```
s/localhost/192.168.1.100/g
EOF

# 应用脚本
sed -f /tmp/process.sed /tmp/config.conf
```

第四部分：实践篇 – awk

4.1 awk 基础

awk 是一种编程语言，专门用于处理结构化文本数据。它把每一行按分隔符拆分成多个字段，然后可以对字段进行各种操作。

基本语法：

```
awk [选项] 'pattern { action }' 文件
```

核心概念：

概念	说明
记录 (Record)	默认是一行
字段 (Field)	一行中被分隔符分开的各个部分
分隔符	默认是空白字符（空格或制表符）

4.2 awk 内置变量

变量	含义
<code>\$0</code>	当前整行内容
<code>\$1, \$2, ...</code>	第 1、2、... 个字段
<code>\$NF</code>	最后一个字段
<code>\$(NF-1)</code>	倒数第二个字段
<code>NF</code>	当前行的字段数量
<code>NR</code>	当前行号（全局）
<code>FNR</code>	当前文件的行号
<code>FS</code>	输入字段分隔符（默认空白）
<code>OFS</code>	输出字段分隔符（默认空格）
<code>RS</code>	输入记录分隔符（默认换行）
<code>ORS</code>	输出记录分隔符（默认换行）
<code>FILENAME</code>	当前文件名

4.3 awk 程序结构

```
awk 'BEGIN { 初始化操作 }
     pattern { 对匹配行的操作 }
     END { 结束时的操作 }' 文件
```

部分	执行时机	典型用途
BEGIN	读取文件前，只执行一次	设置变量、打印表头
主体	对每一行执行	处理数据
END	读取完所有行后，只执行一次	汇总统计、打印结尾

4.4 awk 常用选项

选项	作用	示例
-F	指定字段分隔符	<code>awk -F: '{print \$1}' /etc/passwd</code>
-v	定义变量	<code>awk -v name="Tom" '{print name}'</code>
-f	从文件读取 awk 脚本	<code>awk -f script.awk data.txt</code>

4.5 awk 实战练习

准备测试数据

```
# 创建成绩表
cat > /tmp/scores.txt << 'EOF'
Name      Chinese Math    English
Alice     90      85      92
Bob       78      95      88
Charlie   88      76      90
David     92      88      85
Eve       85      90      95
EOF

# 创建访问日志
cat > /tmp/access.log << 'EOF'
192.168.1.1 - - [01/Jan/2024:10:00:00] "GET /index.html" 200 1234
192.168.1.2 - - [01/Jan/2024:10:00:01] "POST /api/login" 200 567
EOF
```

```
192.168.1.1 - - [01/Jan/2024:10:00:02] "GET /about.html" 200 2345
10.0.0.1 - - [01/Jan/2024:10:00:03] "GET /admin" 403 123
192.168.1.3 - - [01/Jan/2024:10:00:04] "GET /index.html" 200 1234
192.168.1.1 - - [01/Jan/2024:10:00:05] "GET /contact.html" 404 0
192.168.1.2 - - [01/Jan/2024:10:00:06] "GET /index.html" 200 1234
EOF
```

练习 1：基础字段提取

```
# 打印整行
awk '{print $0}' /tmp/scores.txt

# 打印第一列（姓名）
awk '{print $1}' /tmp/scores.txt

# 打印第一列和第三列
awk '{print $1, $3}' /tmp/scores.txt

# 打印最后一列
awk '{print $NF}' /tmp/scores.txt

# 打印倒数第二列
awk '{print $(NF-1)}' /tmp/scores.txt

# 打印字段数量和内容
awk '{print NF, $0}' /tmp/scores.txt
```

练习 2：指定分隔符

```
# 用冒号分隔，打印用户名和 shell
awk -F: '{print $1, $NF}' /etc/passwd | head -5

# 用多个字符作为分隔符
echo "a:blc:d" | awk -F'[:|\]' '{print $1, $2, $3, $4}'

# 使用正则作为分隔符
echo "a1b2c3d" | awk -F'[0-9]' '{print $1, $2, $3}'
```

自定义输出分隔符

```
awk -F: 'BEGIN{OFS=","} {print $1, $3, $NF}' /etc/passwd | head -5
```

输出为制表符分隔

```
awk -F: 'BEGIN{OFS="\t"} {print $1, $3, $NF}' /etc/passwd | head -5
```

练习 3：条件过滤

打印数学成绩大于 85 的学生（跳过标题行）

```
awk 'NR>1 && $3>85 {print $1, $3}' /tmp/scores.txt
```

打印总分大于 260 的学生

```
awk 'NR>1 && ($2+$3+$4)>260 {print $1, $2+$3+$4}' /tmp/scores.txt
```

使用正则匹配

```
awk '/AlicelBob/ {print}' /tmp/scores.txt
```

匹配特定字段

```
awk '$1 ~ /^A/ {print}' /tmp/scores.txt
```

不匹配

```
awk '$1 !~ /^A/ {print}' /tmp/scores.txt
```

打印 200 状态码的访问

```
awk '$9==200 {print}' /tmp/access.log
```

打印非 200 状态码的访问

```
awk '$9!=200 {print $1, $7, $9}' /tmp/access.log
```

练习 4：计算和统计

计算每个学生的总分和平均分

```
awk 'NR>1 {  
    total = $2 + $3 + $4  
    avg = total / 3  
    printf "%s: 总分=%d, 平均=%.2f\n", $1, total, avg  
}' /tmp/scores.txt
```

```
# 计算所有学生的语文平均分
awk 'NR>1 {sum+=$2; count++} END {printf "语文平均分: %.2f\n",
sum/count}' /tmp/scores.txt

# 统计访问日志中各 IP 的访问次数
awk '{ip[$1]++} END {for(i in ip) print i, ip[i]}' /tmp/access.log

# 按访问次数排序
awk '{ip[$1]++} END {for(i in ip) print ip[i], i}' /tmp/access.log |
sort -rn

# 统计各状态码的数量
awk '{code[$9]++} END {for(c in code) print c, code[c]}'
/tmp/access.log

# 统计总流量（最后一个字段）
awk '{sum+=$NF} END {print "总流量:", sum, "bytes"}' /tmp/access.log
```

练习 5：BEGIN 和 END

```
# 打印带表头的报表
awk 'BEGIN {
    print "=====
    print "          学生成绩报表"
    print "=====
    printf "%-10s %-8s %-8s %-8s %-8s\n", "姓名", "语文", "数学", "英
语", "总分"
    print "-----"
}
NR>1 {
    total = $2 + $3 + $4
    printf "%-10s %-8d %-8d %-8d %-8d\n", $1, $2, $3, $4, total
    sum += total
    count++
}
END {
    print "-----"
    printf "平均总分: %.2f\n", sum/count
    print "=====
}' /tmp/scores.txt
```


练习 6：数组的使用

```
# 统计每个 IP 的总流量
awk '{
    ip[$1] += $NF
}
END {
    for(i in ip) {
        printf "%s: %d bytes\n", i, ip[i]
    }
}' /tmp/access.log

# 统计各页面的访问次数
awk '{
    page[$7]++
}
END {
    for(p in page) {
        print page[p], p
    }
}' /tmp/access.log | sort -rn

# 找出访问最多的 IP
awk '{
    ip[$1]++
}
END {
    max = 0
    for(i in ip) {
        if(ip[i] > max) {
            max = ip[i]
            maxip = i
        }
    }
    print "访问最多的 IP:", maxip, "次数:", max
}' /tmp/access.log
```

练习 7：字符串函数

函数	作用
<code>length(s)</code>	字符串长度
<code>substr(s, start, len)</code>	截取子串
<code>index(s, target)</code>	查找位置
<code>split(s, arr, sep)</code>	分割字符串到数组
<code>sub(regex, replacement, target)</code>	替换第一个匹配
<code>gsub(regex, replacement, target)</code>	替换所有匹配
<code>tolower(s)</code>	转小写
<code>toupper(s)</code>	转大写

字符串长度

```
echo "hello world" | awk '{print length($0)}'
```

截取子串

```
echo "hello world" | awk '{print substr($0, 1, 5)}'
```

分割字符串

```
echo "a,b,c,d" | awk '{
    n = split($0, arr, ",")
    for(i=1; i<=n; i++) print arr[i]
}'
```

替换

```
echo "hello world world" | awk '{gsub(/world/, "awk"); print}'
```

大小写转换

```
echo "Hello World" | awk '{print tolower($0)}'
echo "Hello World" | awk '{print toupper($0)}'
```

练习 8：条件语句和循环

```

# if-else
awk 'NR>1 {
    if($3 >= 90) grade="A"
    else if($3 >= 80) grade="B"
    else if($3 >= 70) grade="C"
    else if($3 >= 60) grade="D"
    else grade="F"
    print $1, "数学成绩:", $3, "等级:", grade
}' /tmp/scores.txt

# for 循环 - 打印每个字段
echo "a b c d e" | awk '{
    for(i=1; i<=NF; i++) {
        print "Field", i, "=", $i
    }
}'

# while 循环
awk 'BEGIN {
    i = 1
    while(i <= 5) {
        print "Number:", i
        i++
    }
}'

```

练习 9：实际应用场景

```

# 场景 1: 分析 /etc/passwd
# 统计各 shell 的用户数量
awk -F: '{shell[$NF]++} END {for(s in shell) print shell[s], s}'
/etc/passwd | sort -rn

# 找出 UID 大于 1000 的普通用户
awk -F: '$3>=1000 && $3<65534 {print $1, $3}' /etc/passwd

# 场景 2: 处理 CSV 文件
cat > /tmp/sales.csv << 'EOF'
date,product,quantity,price
2024-01-01,Apple,100,2.5

```

```
2024-01-01,Banana,50,1.5
2024-01-02,Apple,80,2.5
2024-01-02,Orange,120,3.0
2024-01-03,Banana,60,1.5
EOF
```

```
# 计算销售额
```

```
awk -F, 'NR>1 {
    revenue = $3 * $4
    product[$2] += revenue
    total += revenue
}
END {
    print "各产品销售额:"
    for(p in product) printf "  %s: $%.2f\n", p, product[p]
    printf "总销售额: $%.2f\n", total
}' /tmp/sales.csv
```

```
# 场景 3: 处理 df 输出
```

```
df -h | awk 'NR>1 {
    # 去掉 % 号
    gsub(/%/,"", $5)
    if($5 > 50) {
        printf "警告: %s 使用率 %s%%\n", $6, $5
    }
}'
```

```
# 场景 4: 处理 top/ps 输出, 找高 CPU 进程
```

```
ps aux | awk 'NR>1 && $3>1 {print $3"%", $11}' | sort -rn | head -5
```

第五部分：综合实战

5.1 三剑客协作流程

```
# 场景：分析 Web 访问日志，找出访问量最高的 IP 及其访问详情
```

```
# 1. grep 过滤特定类型的请求
```

```
grep 'GET' /tmp/access.log |
```

```
# 2. awk 提取 IP 和页面
awk '{print $1, $7}' |

# 3. 统计每个 IP 访问的不同页面
sort | uniq -c | sort -rn

# 场景：批量修改配置文件

# 1. grep 找出需要修改的配置文件
grep -rI 'old_value' /etc/myapp/ |

# 2. xargs 配合 sed 批量替换
xargs sed -i 's/old_value/new_value/g'
```

5.2 实战练习：日志分析

```
# 创建模拟的 Nginx 日志
cat > /tmp/nginx.log << 'EOF'
192.168.1.100 - - [15/Jan/2024:10:00:01 +0800] "GET /index.html
HTTP/1.1" 200 1234 "-" "Mozilla/5.0"
192.168.1.101 - - [15/Jan/2024:10:00:02 +0800] "POST /api/login
HTTP/1.1" 200 567 "-" "curl/7.68.0"
192.168.1.100 - - [15/Jan/2024:10:00:03 +0800] "GET /css/style.css
HTTP/1.1" 200 890 "-" "Mozilla/5.0"
10.0.0.50 - - [15/Jan/2024:10:00:04 +0800] "GET /admin/index.html
HTTP/1.1" 403 123 "-" "Mozilla/5.0"
192.168.1.102 - - [15/Jan/2024:10:00:05 +0800] "GET /index.html
HTTP/1.1" 200 1234 "-" "Mozilla/5.0"
192.168.1.100 - - [15/Jan/2024:10:00:06 +0800] "GET /images/logo.png
HTTP/1.1" 200 5678 "-" "Mozilla/5.0"
192.168.1.101 - - [15/Jan/2024:10:00:07 +0800] "GET /api/users
HTTP/1.1" 500 0 "-" "curl/7.68.0"
192.168.1.100 - - [15/Jan/2024:10:00:08 +0800] "GET /contact.html
HTTP/1.1" 404 0 "-" "Mozilla/5.0"
EOF

# 任务 1：统计各 IP 的访问次数，按次数降序排列
awk '{print $1}' /tmp/nginx.log | sort | uniq -c | sort -rn

# 任务 2：统计各状态码的数量
```

```
awk '{print $9}' /tmp/nginx.log | sort | uniq -c | sort -rn
```

任务 3: 找出所有 404 和 500 错误

```
grep -E '" (404|500) ' /tmp/nginx.log
```

任务 4: 统计各 URL 的访问次数 (只统计 HTML 页面)

```
grep '\.html' /tmp/nginx.log | awk '{print $7}' | sort | uniq -c | sort -rn
```

任务 5: 计算总流量

```
awk '{sum+=$10} END {printf "总流量: %.2f KB\n", sum/1024}' /tmp/nginx.log
```

任务 6: 找出访问 /admin 的 IP

```
grep '/admin' /tmp/nginx.log | awk '{print $1}' | sort -u
```

任务 7: 统计每小时的访问量

```
awk -F'[/: ]' '{hour[$7]++} END {for(h in hour) print h":00", hour[h]}' /tmp/nginx.log | sort
```

第六部分：配图

正则表达式元字符速查表

字符匹配		
元字符	含义	示例
.	任意单个字符	a.c → abc, a1c
[]	括号内任意一个字符	[abc] → a 或 b 或 c
[^]	不在括号内的字符	[^abc] → d, e, 1...
[a-z]	范围内的字符	任意小写字母
[0-9]	数字字符	0, 1, 2, ..., 9
\\	转义特殊字符	\\. → 匹配真正的点

位置锚定		
元字符	含义	示例
^	行首	^hello → 以hello开头
\$	行尾	world\$ → 以world结尾
^\$	空行	匹配空白行
\\<	词首	\\<the → the, them
\\>	词尾	the\\> → the, bathe
\\b	词边界	\\bthe\\b → 只匹配the

次数匹配		
BRE	ERE	含义
*	*	前一个字符 0次或多次
\\+	+	前一个字符 1次或多次
\\?	?	前一个字符 0次或1次
\\{n\\}	{n}	前一个字符恰好 n次
\\{n,\\}	{n,}	前一个字符至少 n次
\\{n,m\\}	{n,m}	前一个字符 n到m次

示例:
ab*c → ac, abc, abbc, abbbc...
ab+c → abc, abbc (不匹配ac)
ab?c → ac, abc
a{3} → aaa

分组与引用 / 或		
BRE	ERE	含义
\\(\\)	()	分组, 捕获匹配内容
\\1, \\2	\\1, \\2	引用第n个分组的内容
\\		或 (二选一)

分组示例:
(ab)+ → ab, abab, ababab...
(..)\\1 → aa, bb, cc (重复字符)

或示例:
cat|dog → cat 或 dog
(red|blue) car → red car 或 blue car

sed替换示例:
s/(\\w+) (\\w+)/\\2 \\1/
将 "hello world" 变成 "world hello"

使用提示
BRE vs ERE
• BRE (基本): grep, sed 默认
• ERE (扩展): grep -E, sed -r, awk
建议使用扩展正则更清晰 grep -E 'pattern' file sed -r 's/pattern/replace/' file
常用字符类
[[[:alpha:]] 字母
[[[:digit:]] 数字
[[[:alnum:]] 字母和数字
[[[:space:]] 空白字符
[[[:upper:]] 大写字母
[[[:lower:]] 小写字母
常见正则示例
匹配IP [0-9]+\\. [0-9]+\\. [0-9]+\\. [0-9]+
匹配邮箱 [a-zA-Z0-9_ %+-]@[a-zA-Z0-9.-]+
匹配手机号 1[3-9][0-9]{9}
匹配日期 [0-9]{4}-[0-9]{2}-[0-9]{2}
⚠ 注意事项
• 特殊字符需要转义: . * [] ^ \$ \ 等
• 正则默认贪婪匹配, 尽可能多匹配
• 测试正则时先用 echo 和 grep 验证
• sed -i 修改文件前先备份!

第七部分：课后练习

练习一：grep 基础（必做）

1. 在 `/etc/passwd` 中找出所有包含 `root` 的行
2. 找出所有使用 `/bin/bash` 作为 shell 的用户
3. 找出所有不能登录的用户（shell 为 `nologin` 或 `false`）
4. 统计 `/var/log/messages` 中 `ERROR` 出现的次数（忽略大小写）
5. 递归搜索 `/etc` 目录下所有包含 `127.0.0.1` 的文件

练习二：sed 实战（必做）

1. 将文件中所有的 `localhost` 替换为 `192.168.1.100`
2. 删除文件中的空行和注释行（以 `#` 开头）
3. 在文件的第 5 行后插入一行 `# This is a new line`
4. 将 `/etc/passwd` 中的 `root` 用户的 shell 从 `/bin/bash` 改为 `/bin/zsh`（复制文件后操作）
5. 提取 IP 地址并加上方括号，如 `[192.168.1.1]`

练习三：awk 统计（必做）

1. 打印 `/etc/passwd` 的第 1 列和最后一列
2. 统计 `/etc/passwd` 中有多少用户使用 `/bin/bash`
3. 计算一个成绩文件中所有学生的平均分
4. 统计一个日志文件中各 IP 的访问次数，按次数降序排列
5. 生成一个格式化的报表，包含表头、数据和汇总

练习四：综合实战（选做）

场景：分析 Nginx 访问日志

1. 统计访问量最高的 10 个 IP

2. 统计访问量最高的 10 个 URL
 3. 统计各 HTTP 状态码的数量
 4. 找出所有返回 500 错误的请求
 5. 计算总流量（字节数）
 6. 统计每小时的访问量
-

第八部分：课程总结

核心知识点回顾

正则表达式是描述文本模式的语言，核心元字符包括 `.`（任意字符）、`*`（0次或多次）、`^`（行首）、`$`（行尾）、`[]`（字符类）。BRE 和 ERE 的区别主要在于特殊字符是否需要转义。

`grep` 的核心用途是搜索过滤，常用选项包括 `-i`（忽略大小写）、`-v`（反向匹配）、`-n`（显示行号）、`-r`（递归搜索）、`-E`（扩展正则）。

`sed` 的核心用途是流式编辑，最常用的是 `s/old/new/g` 替换命令。`-i` 选项直接修改文件，使用前务必备份。

`awk` 的核心用途是按列处理和统计分析。`$1` 到 `$NF` 表示各个字段，`NR` 是行号，`NF` 是字段数。`BEGIN` 和 `END` 块分别在开始和结束时执行一次。

速记口诀

`grep` 找，`sed` 改，`awk` 算。搜索用 `grep`，替换用 `sed`，统计用 `awk`。三剑客经常组合使用，管道串联威力更大。